WebVella ERP: Business Rules and Semantics

How the platform behaves, written as plain rules

How to read this

These are the rules WebVella ERP enforces, written in plain language. They come from the source code, not from product documentation. Each rule names the file and method it comes from, so you can check it there.

- Rules are numbered by area. For example **Q4** is rule 4 under *EQL queries*.
- An *entity* is a metadata-defined table (for example `task` or `account`). Its rows are *records*, stored in a PostgreSQL table named `rec_<entity>`.
- A *relation* links two entities through two GUID fields. It is 1:1, 1:N or N:N. N:N links are stored in a table named `rel_<relation>`.
- A *hook* is plugin code that runs before (*pre*) or after (*post*) a record operation. A pre-hook can add errors and stop the operation.
- *EQL* is the ERP's SQL-like query language. It is compiled to PostgreSQL.
- The *system user* is a built-in account used for background and setup work.
- File paths are relative to the repository root.

---

1. Entities and fields (E)

E1. Names are lowercase snake_case, 2–63 characters. Entity, field and relation names must start with a letter, use only `a-z`, `0-9` and `_`, and not end with `_`. Two underscores in a row are not allowed. Leading and trailing spaces are trimmed first. *Source: `WebVella.Erp/Api/Models/ValidationUtility.cs (ValidationUtility::ValidateName, NAME_VALIDATION_PATTERN).`*

E2. The 63-character cap is PostgreSQL's identifier limit. Entity and field names longer than 63 characters are rejected with "Should be up to 63 chars". `ValidateName` refuses to run with a larger maximum. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::ValidateEntity, ValidateFields); ValidationUtility.cs (ValidateName).`*

E3. Entity names are unique, and an entity needs an id. Creating an entity whose name already exists fails ("Entity with such Name exists already!"). Updating an entity that does not exist fails. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::ValidateEntity).`*

E4. Every entity has exactly one GUID `id` field. A new entity always gets a required, unique, searchable system GUID field named `id`. A field list with no GUID field, or more than one, is rejected. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::CreateEntityDefaultFields, ValidateFields).`*

E5. Audit fields can be added automatically. When an entity is created without the "only id field" option, it also gets `created_by`, `last_modified_by`, `created_on` and `last_modified_on`. The two date fields default to the current time. All are marked as system fields. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::CreateEntityDefaultFields).`*

E6. New entities are admin-only by default. If no role lists are passed, read, create, update and delete are each granted to the Administrator role only. A missing icon becomes `fa fa-database`. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::CreateEntity, ValidateEntity).`*

E7. Field ids and names are unique within an entity. A second field with the same id or the same name is rejected. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::ValidateField).`*

E8. A required field must have a default value. Most field types (auto-number, currency, email, file, HTML, image, text, number, select and others) need a default when `Required` is set, so existing records can be filled. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::ValidateField).`*

E9. Some field types have extra checks. Date and datetime fields need a format. Select and multiselect fields need at least one option, and option values must not repeat. A unique GUID field must have "generate new id"

switched on. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::ValidateField)`.*

E10. A field used by a relation cannot be deleted. If the field is the origin or target field of any relation, deletion fails and names the relation. *Source: `WebVella.Erp/Api/EntityManager.cs (EntityManager::DeleteField)`.*

E11. Unique and searchable fields get database indexes. A unique field gets a unique constraint named `idx_u_<entity>_<field>`. A searchable field gets an index named `idx_s_<entity>_<field>`. *Source: `WebVella.Erp/Database/DbRecordRepository.cs (DbRecordRepository::CreateRecordField)`.*

---

2. Relations (R)

R1. Both ends of a relation must be GUID fields that exist. The origin and target entities and fields must exist, and both fields must be GUID fields. *Source: `WebVella.Erp/Api/EntityRelationManager.cs (EntityRelationManager::ValidateRelation)`.*

R2. The "one" side of a relation must be required and unique. For 1:1 and N:N, both fields must be required and unique. For 1:N, the origin field must be required and unique. *Source: `WebVella.Erp/Api/EntityRelationManager.cs (EntityRelationManager::ValidateRelation)`.*

R3. A 1:1 relation to the same entity needs two different fields. Using the same field as both origin and target is rejected. *Source: `WebVella.Erp/Api/EntityRelationManager.cs (EntityRelationManager::ValidateRelation)`.*

R4. A relation's shape is fixed after creation. On update, the relation type and the origin/target entities and fields cannot change. *Source: `WebVella.Erp/Api/EntityRelationManager.cs (EntityRelationManager::ValidateRelation)`.*

R5. Relation names follow the same naming rule as entities. See E1. Creating, updating or deleting relations needs meta permission (see S4). *Source: `WebVella.Erp/Api/EntityRelationManager.cs (EntityRelationManager::ValidateRelation, Create)`.*

---

3. Record CRUD (C)

C1. A record gets a new GUID if it has none. An `id` may be passed as a GUID or a string. Any other type is rejected. `Guid.Empty` is never accepted as a record id. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::CreateRecord)`.*

C2. Updates must carry the record id, and the record must exist. An update without `id` fails with "Missing ID field". An unknown id fails with "Record with such Id is not found". *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::UpdateRecord)`.*

C3. Auto-number values are always generated by the database. Any value sent for an auto-number field is ignored on create and on update. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::CreateRecord, UpdateRecord)`.*

C4. Encrypted password fields are stored as an MD5 hash. On update, a `null` password is skipped, so the old password is kept. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::ExtractFieldValue, UpdateRecord)`; `WebVella.Erp/Utilities/PasswordUtil.cs (GetMd5Hash)`.*

C5. Related values are set with `$relation.field` keys, one level deep only. A key such as `$project_nn_task.id` links the record through that relation. `$$` walks the relation from target to origin. Deeper paths, unknown relations and multiselect relation fields are rejected. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::CreateRecord, UpdateRecord)`.*

C6. A relation value must match exactly one related record. If no related record matches, or more than one does, the operation fails. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::CreateRecord, UpdateRecord)`.*

C7. Deleting a record also deletes its files. Files referenced by the record's file fields are removed from file storage before the row is deleted. Image fields are not included in this cleanup. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::DeleteRecord)`.*

C8. Pre-hooks can veto, post-hooks run after success. Create, update and delete run the entity's pre-hooks first. If they add any error, the operation stops. Post-hooks run only after the change is saved. A `RecordManager` built with `executeHooks: false` skips hooks. *Source: `WebVella.Erp/Api/RecordManager.cs (CreateRecord, UpdateRecord, DeleteRecord, constructor)`.*

---

4. Security and permissions (S)

S1. Record access is granted per role, per entity, per action. Each entity has four role lists: read, create, update, delete. A user may act if any of their roles is in the list. *Source: `WebVella.Erp/Api/SecurityContext.cs (SecurityContext::HasEntityPermission)`.*

S2. Anonymous callers act as the Guest role. With no current user, access is allowed only if the Guest role id is in the entity's list for that action. *Source: `WebVella.Erp/Api/SecurityContext.cs (SecurityContext::HasEntityPermission)`; `WebVella.Erp/Api/Definitions.cs (SystemIds.GuestRoleId)`.*

S3. The system user can do everything with records. Entity permission checks always pass for `SystemIds.SystemUserId`. Plugins use `SecurityContext.OpenSystemScope()` for setup and background work. *Source: `WebVella.Erp/Api/SecurityContext.cs (HasEntityPermission, OpenSystemScope)`.*

S4. Only administrators may change metadata. Creating, updating or deleting entities, fields and relations needs the Administrator role. Anonymous callers never have it. *Source: `WebVella.Erp/Api/SecurityContext.cs (SecurityContext::HasMetaPermission)`; `WebVella.Erp/Api/EntityManager.cs (CreateEntity, DeleteEntity, DeleteField)`.*

S5. Each CRUD call checks its own permission. Create, update, delete and find each check the matching right and return "Access denied" if it is missing. A `RecordManager` created with `ignoreSecurity: true` skips these checks. *Source: `WebVella.Erp/Api/RecordManager.cs (CreateRecord, UpdateRecord, DeleteRecord, Find)`.*

S6. Record counts are not permission-checked. `RecordManager.Count` runs the count without a read-permission check, unlike `Find`. *Source: `WebVella.Erp/Api/RecordManager.cs (RecordManager::Count)`.*

S7. The current user is scoped per async flow. `OpenScope(user)` pushes a user onto a stack held in `AsyncLocal`. Disposing the scope pops it, so nested scopes restore the previous user. *Source: `WebVella.Erp/Api/SecurityContext.cs (SecurityContext::OpenScope)`.*

S8. Usernames and emails are unique, and emails must be valid. A new user needs a username, an email and a password. A username or email already used by another user is rejected. *Source: `WebVella.Erp/Api/SecurityManager.cs (SecurityManager::SaveUser, IsValidEmail)`.*

---

5. Hooks (H)

H1. Hooks attach to one entity by name. A class marked `[HookAttachment("task")]` runs only for the `task` entity. Hooks are found by scanning loaded assemblies at startup. *Source: `WebVella.Erp/Hooks/HookManager.cs (HookManager::RegisterHooks, GetHookedInstances)`.*

H2. Higher priority runs first. Hooks for the same key run in descending `Priority` order. The default priority is 10. The search-index hooks of the Next plugin use `int.MinValue`, so they run last. *Source: `WebVella.Erp/Hooks/HookManager.cs (GetHookedInstances)`; `WebVella.Erp/Hooks/HookAttachmentAttribute.cs`; `WebVella.Erp.Plugins.Next/Hooks/Api/TaskHook.cs`.*

H3. There are hooks for search, create, update, delete and N:N links. Record hooks cover pre/post search, create, update and delete. Relation hooks cover pre/post create and delete of N:N links. *Source: `WebVella.Erp/Hooks/RecordHookManager.cs (RecordHookManager)`.*

---

6. EQL queries (Q)

Q1. A query is `SELECT … FROM … [WHERE …] [ORDER BY …] [PAGE n PAGESIZE m]`. Arguments are written `@name` and must be supplied as parameters. A missing parameter is an error. *Source: `WebVella.Erp/Eql/EqlGrammar.cs`; `WebVella.Erp/Eql/EqlBuilder.Sql.cs (EqlBuilder::ProcessExpressionNode)`.*

Q2. `PAGE` and `PAGESIZE` go together and must be positive. Using only one is an error. They become `LIMIT m OFFSET (n-1)*m`. *Source: `WebVella.Erp/Eql/EqlBuilder.Sql.cs (EqlBuilder::BuildSql)`.*

Q3. Related data is selected with `$relation.field` (or `$$` for the reverse direction). Related rows come back as a JSON list inside each record. *Source: `WebVella.Erp/Eql/EqlGrammar.cs`; `WebVella.Erp/Eql/EqlBuilder.Sql.cs (BuildFieldsSql)`.*

Q4. `WHERE` may only use first-level relation fields. The first operand of a comparison must be a field name. *Source: `WebVella.Erp/Eql/EqlBuilder.Sql.cs (ProcessExpressionOperandNode, ProcessExpressionNode)`.*

Q5. `CONTAINS` and `STARTSWITH` ignore case. They become `ILIKE '%x%'` and `ILIKE 'x%'`. The regex operators `~`, `~*`, `!~`, `!~*` are passed to PostgreSQL as is. *Source: `WebVella.Erp/Eql/EqlBuilder.Sql.cs (EqlBuilder::ProcessExpressionNode)`.*

Q6. `NULL` only works with `=`, `<>` and `!=`. `= NULL` becomes `IS NULL` and `<> NULL` becomes `IS NOT NULL`. Any other operator with `NULL` is an error. *Source: `WebVella.Erp/Eql/EqlBuilder.Sql.cs (EqlBuilder::ProcessExpressionNode)`.*

Q7. `@@` is full-text search. It uses PostgreSQL `to_tsvector('simple', …)`. A single word is matched as a prefix (`word:*`). Several words, or empty text, use `plainto_tsquery`. *Source: `WebVella.Erp/Eql/EqlBuilder.Sql.cs (EqlBuilder::ProcessExpressionNode)`.*

Q8. `ORDER BY` fields must exist on the queried entity. The direction may come from a parameter, but it must be `ASC` or `DESC`. *Source: `WebVella.Erp/Eql/EqlBuilder.Sql.cs (BuildSql)`; `WebVella.Erp/Eql/EqlBuilder.cs (BuildOrderByNode)`.*

Q9. Results include a total count by default. `EqlSettings.IncludeTotal` is `true`, which adds `COUNT(*) OVER()` to the query. `Distinct` is `false` by default. *Source: `WebVella.Erp/Eql/EqlSettings.cs`; `WebVella.Erp/Eql/EqlBuilder.Sql.cs (BuildSql, BuildFieldsSql)`.*

Q10. EQL respects security and hooks. Pre-search hooks can change the query or add errors before SQL is built. When results are read, every selected field's entity (main or related) is checked for read permission, and a missing right throws "No access to entity". Post-search hooks see the results. *Source: `WebVella.Erp/Eql/EqlBuilder.cs (Build)`; `WebVella.Erp/Eql/EqlCommand.cs (Execute, ConvertJObjectToEntityRecord)`.*

Q11. A query may run for up to 10 minutes. The SQL command timeout is 600 seconds. *Source: `WebVella.Erp/Eql/EqlCommand.cs (EqlCommand::Execute)`.*

---

7. Background jobs and schedules (J)

J1. Background jobs run only when enabled. If `EnableBackgroundJobs` is off, the job and schedule loops return at once. *Source: `WebVella.Erp/ERPService.cs (InitializeBackgroundJobs)`; `WebVella.Erp/Jobs/JobManager.cs (ProcessJobsAsync)`; `WebVella.Erp/Jobs/SheduleManager.cs (ProcessSchedulesAsync)`.*

J2. Jobs that were running at startup are marked aborted. When the job manager starts, every job still in `Running` state becomes `Aborted`, with "aborted by" set to `Guid.Empty` (system). *Source: `WebVella.Erp/Jobs/JobManager.cs (JobManager constructor)`.*

J3. At most 20 jobs run at once. Pending jobs are picked by priority (high first), then by creation time (oldest first). *Source: `WebVella.Erp/Jobs/JobPool.cs (MAX_THREADS_POOL_COUNT)`; `WebVella.Erp/Jobs/JobDataService.cs (GetJobs)`.*

J4. Single-instance job types never overlap. If a job type allows a single instance and one is already in the pool, new jobs of that type wait. *Source: `WebVella.Erp/Jobs/JobManager.cs (ProcessJobsAsync)`; `WebVella.Erp/Jobs/JobPool.cs (HasJobFromTypeInThePool)`.*

J5. Job status moves Pending → Running → Finished or Failed. An exception marks the job `Failed` and stores the error message. An invalid priority falls back to the job type's default, which is `Low` unless the `[Job]` attribute says otherwise. *Source: `WebVella.Erp/Jobs/JobManager.cs (CreateJob)`; `WebVella.Erp/Jobs/JobPool.cs (JobPool::Process)`; `WebVella.Erp/Jobs/JobAttribute.cs`.*

J6. In release builds, processing starts 2 minutes after startup. The hosted job and schedule services wait 120 seconds (not in DEBUG builds) before their first pass. *Source: `WebVella.Erp/Jobs/JobManager.cs (ProcessJobsAsync(CancellationToken))`; `WebVella.Erp/Jobs/SheduleManager.cs (ProcessSchedulesAsync(CancellationToken))`.*

J7. A schedule never starts a job while its last job is unfinished. A schedule plan fires when it is enabled, its start date has passed and its next trigger time is due. If the job it started last time has no finish time yet, no new job is created. *Source: `WebVella.Erp/Jobs/SheduleManager.cs (ProcessSchedulesAsync)`; `WebVella.Erp/Jobs/JobDataService.cs (GetReadyForExecutionScheduledPlans, IsJobFinished)`.*

J8. Schedules are interval, daily, weekly or monthly, in UTC. An interval plan with `IntervalInMinutes <= 0` never triggers. A plan stops when the next trigger would be after its end date. Time windows are minutes after midnight and may wrap past midnight. *Source: `WebVella.Erp/Jobs/SheduleManager.cs (FindIntervalSchedulePlanNextTriggerDate, ProcessSchedulesAsync)`.*

J9. "Trigger now" means in one minute. Triggering a plan manually sets its next trigger time to now plus one minute. *Source: `WebVella.Erp/Jobs/SheduleManager.cs (ScheduleManager::TriggerNowSchedulePlan)`.*

---

8. Recurrence (T)

T1. A new recurrence plan does not repeat. Defaults: type `DoesNotRepeat`, end type `Never`, interval 1, period `Day`, all seven weekdays allowed, monthly repeat by date. *Source: `WebVella.Erp/Recurrence/RecurrencePlan.cs (RecurrencePlan properties)`.*

T2. "Never ends" is calculated five years ahead. Occurrences for an open-ended plan stop five years after the start time. *Source: `WebVella.Erp/Recurrence/RecurrencePlan.cs (CalculateOccurrences, CALCULATION_EXTEND_YEARS)`.*

T3. "End by date" needs a date. If the end type is `Date` and `EndDate` is empty, calculation fails. *Source: `WebVella.Erp/Recurrence/RecurrencePlan.cs (CalculateOccurrences)`.*

T4. "End after N" returns exactly N occurrences when possible. The window grows five years at a time until there are enough, then the list is cut to N. *Source: `WebVella.Erp/Recurrence/RecurrencePlan.cs (CalculateOccurrences)`.*

T5. Monthly and yearly patterns follow the start date. "Monthly by date" repeats on the start day of month. "Monthly by weekday" repeats on the same weekday and week of month (for example the second Tuesday). Custom plans use `Interval` with a period from seconds to years. *Source: `WebVella.Erp/Recurrence/RecurrencePlan.cs (CalculateOccurrences, GetWeekOfMonth)`.*

---

9. Projects, tasks and time logs (P)

P1. A task belongs to exactly one project. Creating a task from the UI with no project, or with more than one, fails ("Project is not specified." / "More than one project is selected."). *Source: `WebVella.Erp.Plugins.Project/Services/TaskService.cs (TaskService::PreCreateRecordPageHookLogic)`.*

P2. A task key is the project abbreviation plus the task number. For example `ABC-12`. It is set after the task is created. *Source: `WebVella.Erp.Plugins.Project/Services/TaskService.cs (TaskService::SetCalculationFields, PostCreateApiHookLogic)`.*

P3. New tasks start with watchers. The owner, the creator and the project owner are added as watchers, without duplicates. *Source: `WebVella.Erp.Plugins.Project/Services/TaskService.cs (TaskService::PostCreateApiHookLogic)`.*

P4. Key and search fields are recalculated after every task update. When the project changes, the pre-update hook removes the old project link. Its “add new link” and “add new project owner as watcher” steps call `RemoveRelationManyToManyRecord`, not `CreateRelationManyToManyRecord`, so they do not add anything. *Source: WebVella.Erp.Plugins.Project/Services/TaskService.cs (TaskService::PostPreUpdateApiHookLogic, PostUpdateApiHookLogic).*

P5. Tasks start automatically on their start date. A single-instance job moves every task in status “not started” with `start_time` today or earlier to status “started”. *Source: WebVella.Erp.Plugins.Project/Jobs/StartTasksOnStartDate.cs; TaskService.cs (GetTasksThatNeedStarting).*

P6. Open-task lists hide closed statuses. Any status flagged `is_closed` is excluded from “due” task lists. *Source: WebVella.Erp.Plugins.Project/Services/TaskService.cs (TaskService::GetTaskQueue).*

P7. A project time log must point at an existing task. For time logs scoped to `projects`, the related task must exist, or creation fails. *Source: WebVella.Erp.Plugins.Project/Services/TimeLogService.cs (TimeLogService::PreCreateApiHookLogic).*

P8. Time logs roll up into task totals. Logged minutes are added to the task’s `x_billable_minutes` or `x_nonbillable_minutes`. Logging time also stops the task’s running timer (`timelog_started_on` becomes empty). *Source: WebVella.Erp.Plugins.Project/Services/TimeLogService.cs (TimeLogService::PreCreateApiHookLogic).*

P9. Deleting a time log subtracts its minutes, never below zero. Related feed items are deleted too. *Source: WebVella.Erp.Plugins.Project/Services/TimeLogService.cs (TimeLogService::PreDeleteApiHookLogic).*

P10. New time logs are billable by default. `TimeLogService.Create` defaults to 0 minutes and billable. *Source: WebVella.Erp.Plugins.Project/Services/TimeLogService.cs (TimeLogService::Create).*

P11. Only the author can delete a time log or a comment. Deleting a comment also deletes its direct replies. *Source: WebVella.Erp.Plugins.Project/Services/TimeLogService.cs (Delete); CommentService.cs (CommentService::Delete).*

P12. A project comment must point at an existing task, and its author starts watching. After a project comment is created, its author is added to the task’s watchers if missing. *Source: WebVella.Erp.Plugins.Project/Services/CommentService.cs (PreCreateApiHookLogic, PostCreateApiHookLogic).*

---

10. CRM and search index (X)

X1. The CRM plugin only installs and patches its data model. It runs versioned patches inside a transaction as the system user. The current version is kept in plugin data. *Source: WebVella.Erp.Plugins.Crm/CrmPlugin._.cs (CrmPlugin::ProcessPatches).*

X2. Plugin patches are all-or-nothing. If any patch fails, the transaction is rolled back and the error is re-thrown. The saved version only moves forward on success. *Source: WebVella.Erp.Plugins.Project/ProjectPlugin._.cs (ProjectPlugin::ProcessPatches).*

X3. Accounts, contacts, cases and tasks keep a search text field. After each create or update, `x_search` is rebuilt from a fixed list of fields. For accounts this includes name, email, phones, address, tax id and website. *Source: WebVella.Erp.Plugins.Next/Hooks/Api/AccountHook.cs; WebVella.Erp.Plugins.Next/Configuration.cs (AccountSearchIndexFields); Services/SearchService.cs (RegenSearchField).*

---

11. Mail (M)

M1. An email needs a valid recipient and a subject. Every recipient address must parse as an email, and the subject must not be empty. At send time, a `cc:` or `bcc:` prefix on a recipient address puts it in CC or BCC. *Source: WebVella.Erp.Plugins.Mail/Api/SmtpService.cs (SmtpService::SendEmail, QueueEmail); Services/SmtpInternalService.cs (SendEmail).*

M2. Queued emails wait as Pending. A queued email gets status `Pending`, 0 retries, normal priority by default, and `ScheduledOn` equal to its creation time. The default sender and reply-to come from the SMTP service. *Source: WebVella.Erp.Plugins.Mail/Api/SmtpService.cs (SmtpService::QueueEmail).*

M3. The queue sends high priority first, then oldest. Pending emails that are due are taken 10 at a time, ordered by priority then scheduled time. Only one queue run happens at a time. *Source: WebVella.Erp.Plugins.Mail/Services/SmtpInternalService.cs (SmtpInternalService::ProcessSmtpQueue).*

M4. The queue runs every 10 minutes. The Mail plugin creates an interval schedule plan (10 minutes, every day, all day) for the single-instance "Process SMTP queue" job. *Source: WebVella.Erp.Plugins.Mail/MailPlugin.cs (SetSchedulePlans); Jobs/ProcessSmtpQueueJob.cs.*

M5. Failed sends are retried, then aborted. Each failure adds one retry and reschedules the email after the service's retry wait. When retries reach the service's maximum, the email is `Aborted`. A missing or disabled SMTP service aborts the email at once. *Source: WebVella.Erp.Plugins.Mail/Services/SmtpInternalService.cs (SmtpInternalService::SendEmail).*

M6. SMTP service settings have hard limits. Port 1–65025, max retries 1–10, retry wait 1–1440 minutes. The name must be unique, and default sender/reply-to must be valid emails. *Source: WebVella.Erp.Plugins.Mail/Services/SmtpInternalService.cs (ValidatePreCreateRecord, ValidatePreUpdateRecord).*

M7. There is always exactly one default SMTP service. Marking a service default clears the flag on all others. Un-marking the current default, or deleting it, is refused. *Source: WebVella.Erp.Plugins.Mail/Services/SmtpInternalService.cs (HandleDefaultServiceSetup); Hooks/Api/SmtpServiceRecordHook.cs (OnPreDeleteRecord).*

M8. Reply-to falls back to the sender. Several reply-to addresses may be separated with `;`. If none is set, replies go to the sender address. *Source: WebVella.Erp.Plugins.Mail/Services/SmtpInternalService.cs (SmtpInternalService::SendEmail).*

---

12. Configuration constraints and defaults (K)

K1. Settings come from `Config.json`, with code fallbacks. `ErpSettings.Initialize` reads the `Settings` section. The table shows the code fallback and the value in the sample site config. *Source: WebVella.Erp/ErpSettings.cs (ErpSettings::Initialize); WebVella.Erp.Site/Config.json.*

Setting	Code default	Site `Config.json`	Rule
`Lang` / `Locale`	`en` / `en-US`	`en` / `en-US`	UI language and culture
`TimeZoneName`	`FLE Standard Time`	same	App time zone
`JsonDateTimeFormat`	`yyyy-MM-ddTHH:mm:ss.fff`	not set	Date format in JSON
`CacheKey`	today as `yyyyMMdd`	empty (uses default)	Static-file cache key
`DevelopmentMode`	`false`	`true`	Developer features on/off
`EnableBackgroundJobs`	`true`	`false`	See J1. The misspelled `EnableBackgroungJobs` is still read as a fallback
`EnableFileSystemStorage`	`false`	`false`	Store files on disk instead of DB
`FileSystemStorageFolder`	`c:\erp-files`	network share path	Folder for disk storage
`EnableCloudBlobStorage`	`false`	not set	Use blob storage
`EmailEnabled` / `EmailSMTPPort`	`false` / `25`	`false` / `25`	Core email settings
`ShowAccounting`	`false`	not set	Show accounting UI

Setting	Code default	Site `Config.json`	Rule
`Jwt:Issuer` / `Jwt:Audience`	`webvella-erp`	`webvella-erp`	API token claims
`Jwt:Key`	built-in demo key	demo key	Change before production
`DefaultSRID`	`4326`	not set	Geography field spatial ref

K2. Built-in limits.

Limit	Value	Source
Name length (entity, field, relation)	2–63 chars	`ValidationUtility.ValidateName`
View/list name length	up to 200 chars, `~` allowed	`ValidationUtility.ValidateViewName`
Concurrent jobs	20	`JobPool.MAX_THREADS_POOL_COUNT`
Job/schedule start delay (release)	120 s	`JobManager`, `ScheduleManager`
EQL command timeout	600 s	`EqlCommand.Execute`
Entity/relation metadata cache	1 hour	`Api/Cache.cs`
Recurrence look-ahead	5 years	`RecurrencePlan.CALCULATION_EXTEND_YEARS`
SMTP port / retries / retry wait	1–65025 / 1–10 / 1–1440 min	`SmtpInternalService`
SMTP queue batch	10 emails	`SmtpInternalService.ProcessSmtpQueue`